

# Database Sharding & Partitioning

Phase 2 · Intermediate · Estimated time: 2 hours

---

## 1. Concept From First Principles

Chapter 1 was explicit that read replicas and connection pooling never solve a write-throughput or total-data-size problem — they only help with reads. This chapter covers the technique that actually does: **sharding**, the practice of splitting your data across multiple independent database instances (shards), each holding a subset of the total rows, so that no single machine has to store or process the entire dataset alone.

It's worth separating two terms that get used loosely and interchangeably in casual conversation but mean distinct things. **Partitioning** is the general idea of splitting a large table into smaller pieces, and can happen entirely within a single database instance (useful for query performance and maintenance even without any scaling need). **Sharding** specifically means partitioning across multiple separate database instances/servers, each an independent, complete database in its own right — sharding is partitioning done specifically for horizontal scale-out. Every shard, by the way, benefits from everything Chapters 1 and 5 covered: each shard can have its own read replicas and connection pool.

### Horizontal vs. vertical partitioning

**Horizontal partitioning** (the far more common meaning when people say "sharding") splits a table by *rows* — users with IDs 1–10 million go to Shard A, users with IDs 10–20 million go to Shard B, and so on, with every shard sharing the identical schema. **Vertical partitioning** splits a table by *columns* instead — rarely-accessed or very large columns (like a lengthy "bio" text field) get moved to a separate table or store, keeping the frequently-accessed columns compact and fast to scan. The two are not mutually exclusive; large systems often do both.

#### Backend engineer analogy

Imagine your Laravel app's **users** table has grown so large that a single MySQL instance chokes under the write load. Sharding is the equivalent of running four separate, identical Laravel + MySQL deployments, each responsible for one-quarter of your users by ID range, with your application code deciding at request time which deployment to talk to based on the incoming user's ID — instead of one giant, overloaded database serving everyone.

The single most consequential decision in sharding is the choice of **shard key** (also called a partition key) — the field used to decide which shard a given row lives on. A poorly chosen shard key creates a **hotspot**: if you shard an orders table by, say, `restaurant_id`, and one wildly popular restaurant chain generates 40% of all orders, that single shard becomes overloaded while others sit mostly idle, defeating the entire point of sharding. A well-chosen shard key spreads load and data as evenly as possible across shards, which usually means choosing a high-cardinality, roughly-uniformly-distributed field like a user ID or a UUID rather than something naturally skewed like a category or region.

Sharding also introduces a genuinely hard new category of problem: **cross-shard queries**. A query like "find all orders placed by user X" is trivial if the shard key is `user_id` (it hits exactly one shard), but a

query like "find the total revenue across all restaurants this month" now has to query every single shard and aggregate the results in the application layer, since no single shard has the complete picture. Similarly, operations that used to be a single-database transaction (like moving money between two accounts) become far harder to do atomically once those two accounts might live on different shards — this is precisely why interview-savvy engineers treat sharding as a last resort, reached for only once caching, replicas, and vertical scaling have genuinely run out of headroom, not a default starting point.

Finally, **resharding** — changing the number of shards or the shard key later, as data grows unevenly — is one of the most operationally painful events in a system's life, often requiring careful, sometimes downtime-inducing data migration. This is why many teams intentionally over-provision shard count upfront (e.g., 1,024 logical shards mapped onto a smaller number of physical database instances that can later be split without changing the logical shard count), a pattern popularized by systems like Vitess (used at YouTube for MySQL sharding) precisely to make resharding a physical, not logical, migration.

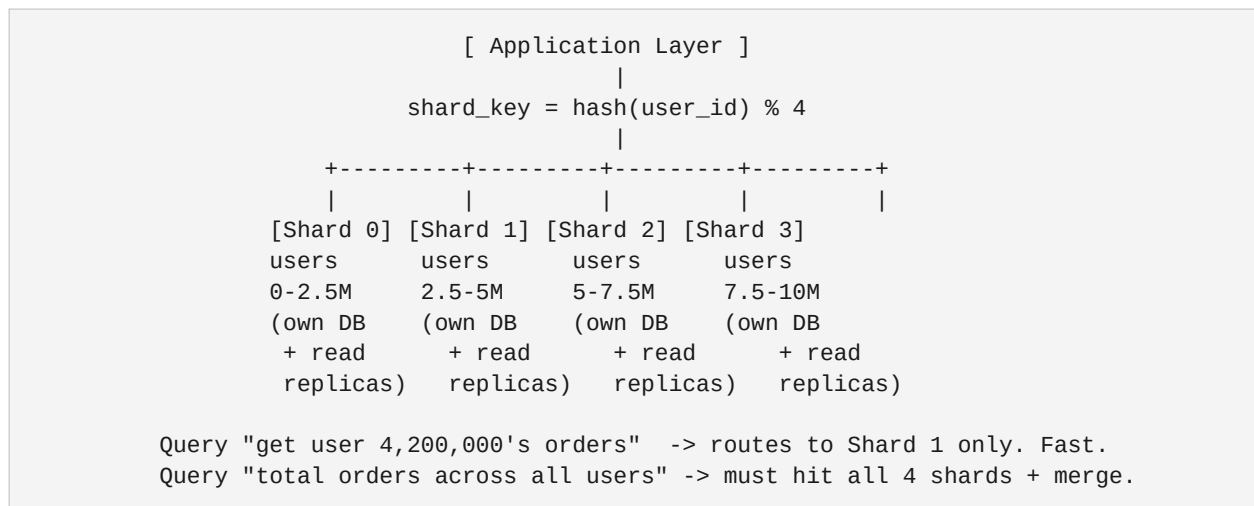
## 2. Real-World Example / Case Study

Flipkart's order and catalog systems, operating at hundreds of millions of products and orders, shard their data across many database instances, typically keyed by something like `user_id` or `seller_id`, so that a query for "this user's order history" stays fast by hitting exactly one shard rather than scanning a monolithic table with billions of rows.

Swiggy's and Zomato's order systems face a genuinely interesting shard-key decision: sharding by `restaurant_id` would create severe hotspots around popular chains during peak lunch/dinner rushes, while sharding by a combination like `city_id` + `user_id` spreads load more evenly, since order volume is more naturally distributed across a large customer base than across a comparatively small number of restaurant accounts — a direct illustration of the hotspot problem this chapter warns about.

Razorpay and PhonePe, needing strong transactional guarantees within a shard but processing enormous transaction volume, typically shard by `merchant_id` or `account_id`, ensuring that a single merchant's or account's transaction history and balance operations stay within one shard (avoiding the cross-shard transaction problem for the most common, latency-sensitive operations), while any cross-shard reporting or reconciliation runs as separate, slower, aggregation jobs rather than live-path queries.

## 3. Diagram



## 4. Trade-offs Table

| Decision                                          | Benefit                                                  | Cost / Risk                                                 |
|---------------------------------------------------|----------------------------------------------------------|-------------------------------------------------------------|
| Shard by user_id / high-cardinality key           | Even load distribution, avoids hotspots                  | Cross-user aggregate queries need fan-out                   |
| Shard by natural category (region, restaurant)    | Intuitive, easy to reason about                          | High risk of hotspots on popular categories                 |
| Over-provision logical shards (e.g. Vitess-style) | Resharding later is a physical move, not a logical remap | More upfront complexity to set up                           |
| Keep related data co-located on one shard         | Enables transactions/joins within that shard             | Requires careful shard-key design up front                  |
| Shard only when truly necessary                   | Avoids premature complexity                              | Risk of scrambling to shard under production pressure later |

## 5. Common Interview Questions

### Q1: "How would you choose a shard key for a multi-tenant SaaS product's data?"

Model approach: Recommend tenant\_id if tenants are numerous and roughly similar in size (even distribution), but flag that a small number of very large 'whale' tenants would create hotspots, in which case you'd need a composite key or special-case very large tenants onto dedicated shards.

### Q2: "How do you run a query that needs data from multiple shards?"

Model approach: Explain the scatter-gather pattern — the application (or a query-routing layer) sends the query to every relevant shard in parallel, then merges/aggregates results in the application layer, and note this is inherently slower and more complex than a single-shard query, which is exactly why shard-key choice matters so much for your most frequent queries.

### Q3: "What makes resharding hard, and how do systems avoid it?"

Model approach: Explain that changing shard count or key means physically moving large amounts of data with minimal downtime, and describe the over-provisioned logical-shard pattern (many logical shards on fewer physical machines, splittable later) as the standard mitigation, citing Vitess/YouTube

as a real example.

#### Q4: "When would you NOT shard, even at large scale?"

Model approach: If vertical scaling, caching, and read replicas still comfortably handle the load, sharding adds operational complexity (cross-shard queries, harder transactions, resharding pain) that isn't yet justified — a senior answer explicitly treats sharding as a last resort reached for only when simpler techniques are provably insufficient.

## 6. Practice Exercises

- 1 Propose a shard key for a ride-hailing app's trips table, and identify one query that would become a slow, cross-shard scatter-gather under your chosen key.
- 2 A social media platform shards posts by post\_id, but 'get all posts by user X' now has to query every shard. Redesign the sharding strategy to fix this, and state what trade-off you're accepting instead.
- 3 Explain, step by step, why moving money between two accounts is easy within one shard but hard across two different shards.

## 7. Curated YouTube Videos

| Language | Video & Channel                                                                                  | Why it helps                                                               |
|----------|--------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------|
| English  | "Database Partitioning & Sharding Explained   Complete System Design Guide"                      | Comprehensive overview of both partitioning types and shard-key selection. |
| English  | "System Design - Sharding   Data Partitioning"                                                   | Focused, example-driven walkthrough of horizontal sharding.                |
| Hindi    | "Database Sharding in System Design Explained in Hindi with Real Life Example" — Shridhar Mankar | Hindi walkthrough with concrete real-life sharding examples.               |
| Hindi    | "Lec-11: Database Partitioning vs Sharding   System Design"                                      | Hindi lecture-style comparison of partitioning vs sharding.                |

## 8. Key Takeaways

- Sharding splits data across multiple independent database instances to scale beyond one machine's limits.
- Partitioning is the general concept; sharding specifically means partitioning across separate servers.
- Horizontal partitioning splits by rows; vertical partitioning splits by columns; both can be combined.
- Shard key choice is the most consequential decision — a poor key creates hotspots that defeat the purpose.
- Cross-shard queries and cross-shard transactions are genuinely harder than their single-database equivalents.
- Resharding later is operationally painful; over-provisioning logical shards up front (Vitess-style) mitigates this.
- Treat sharding as a last resort after caching, replicas, and vertical scaling are proven insufficient.